05. Station 4: Master Conductor (`main.py`)

1. Overview & Purpose

`main.py` is the **Master Conductor & Orchestrator** of the sensing layer.

It ties the entire pipeline together:

1. Boots the background HTTP shipper thread (`start_shipper()`).
2. Activates the network interface card in promiscuous mode via Scapy's `sniff()`.
3. Passes every captured frame into `ja4.process_packet()`.
4. Takes resulting enriched metadata and pushes it into `shipper.enqueue_log()`.
5. Enforces zero packet retention (`store=0`) to eliminate memory leaks.
6. Catches `KeyboardInterrupt` (`Ctrl+C`) to guarantee graceful termination.

2. Complete Source Code

```
python
from scapy.all import sniff
from ja4 import process_packet
from shipper import start_shipper, enqueue_log

def on_packet(packet):
    meta = process_packet(packet)
    if meta:
        enqueue_log(meta)

if __name__ == "__main__":
    print("[INFO] Starting SIEM Network Sensing Layer...")
    start_shipper()
    try:
        sniff(iface="eth0", filter="tcp port 443", prn=on_packet, store=0)
    except KeyboardInterrupt:
        print("\n[INFO] Sensor stopped by user.")
```

3. Function-by-Function Breakdown

1. `on_packet(packet)`

- **Role:** Pipeline callback bridging Feature Extraction (Station 2) and Shipping (Station 3).
- **Execution:**
    1. Invoked by Scapy whenever a packet matching the filter (`tcp port 443`) arrives on the interface.
    2. Passes the raw packet to `process_packet(packet)`. This runs the gatekeeper check, extracts IP/TCP headers, computes the JA4 fingerprint if a ClientHello is present, and looks up the bidirectional session table.
    3. If `meta` is returned (not dropped), calls `enqueue_log(meta)` to push the event into the bounded RAM queue.

2. Main Execution Block (`if __name__ == "__main__":`)

- **Role:** Process lifecycle manager.
- **Execution:**
    1. Displays startup banner: `[INFO] Starting SIEM Network Sensing Layer...`
    2. Calls `start_shipper()` to spin up the background HTTP daemon worker.
    3. Calls `sniff(iface="eth0", filter="tcp port 443", prn=on_packet, store=0)`:
        - `iface="eth0"`: Listens on the primary network interface.
        - `filter="tcp port 443"`: Kernel-level BPF filter ignoring non-TLS traffic.
        - `prn=on_packet`: Directs Scapy to run `on_packet` for every captured frame.
        - `store=0`: **Critical memory safeguard.** Tells Scapy to discard the raw packet from RAM as soon as `on_packet` finishes.
    4. Wraps `sniff()` inside `try...except KeyboardInterrupt:` so pressing `Ctrl+C` prints a clean shutdown confirmation instead of a raw Python traceback.

4. Line-by-Line Explanation

Line	Code	Explanation in Simple Terms
1	`from scapy.all import sniff`	Imports Scapy's high-performance packet capture engine.
2	`from ja4 import process_packet`	Imports the feature extractor from Station 2.
3	`from shipper import start_shipper, enqueue_log`	Imports the background thread launcher and queue producer from Station 3.
5	`def on_packet(packet):`	Defines the bridge callback executed per packet.
6	`meta = process_packet(packet)`	Hands the packet to Station 2 for gatekeeping, parsing, and JA4 calculation.
7-8	`if meta: enqueue_log(meta)`	If the packet was not dropped, enqueues the dictionary for asynchronous shipping.
10	`if __name__ == "__main__":`	Ensures the sniffer only starts when executed as the main script.
11	`print("[INFO] Starting SIEM...")`	Logs startup to console.
12	`start_shipper()`	Launches the background worker thread before sniffing begins.
13-15	`try: sniff(...)`	Activates promiscuous packet capture with BPF filter and `store=0` memory guard.
16-17	`except KeyboardInterrupt: print(...)`	Traps `Ctrl+C` for an orderly exit.

5. Variables & System Mapping

Variable	Type	Scope	Role in `main.py`	Downstream Mapping (Backend / System)
`packet`	`scapy.Packet`	Argument to `on_packet`	The live packet captured from the wire.	Internal to capture pipeline. Discarded immediately after parsing.
`meta`	`dict`	Local	The fully enriched metadata dictionary.	Enqueued and shipped to backend `LogEvent` document collection.
`iface`	`str`	Argument to `sniff()`	Network interface identifier (e.g. `eth0`).	Internal OS configuration.
`filter`	`str`	Argument to `sniff()`	BPF filter string (`tcp port 443`).	Internal kernel packet filter.
`store=0`	`int`	Argument to `sniff()`	Disables in-memory packet buffering.	Prevents process out-of-memory crashes on long runs.